

SD Basic - String Functions

Measuring, finding, extracting, trimming, building and changing the case of text.

This page covers the functions that work on a string as text. The functions that work on a string as a **structure** — fields, values and subvalues held together by marks — are in SD Basic - Dynamic Arrays, and the ones that convert between internal and external form are in SD Basic - Data Conversion. The line between the three is not always obvious, so each page points at the others where they meet.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

Strings, and what counts as a character

Everything in SD is a string. There is no separate character type and no length declaration — a variable holds as much as you put in it.

The mark characters are ordinary characters to every function on this page. `@fm`, `@vm` and `@sm` occupy one byte each and are counted, indexed, extracted and replaced like any other:

```
len(1 : @vm : 2)      ;* 3 - the two digits and the mark between them
len(@fm)              ;* 1
```

That is convenient when you want it and a trap when you do not. A `len()` used to size a display field will include the marks; an `index()` searching for a character may find one inside a mark-delimited structure you did not mean to look at. Where you want structure rather than text, use the dynamic-array functions.

Case conversion leaves marks alone: `upcase(@vm)` is still `@vm`.

Measuring and counting

```
len(string)
count(string, substring)
dcount(string, delimiter)
```

`len()` the number of characters. `len('hello')` is `5`, `len('')` is `0`

`count()` how many times *substring* occurs

`dcount()` how many **fields** *delimiter* divides the string into — normally one more than `count()`

THE DIFFERENCE IN PRACTICE

EXPRESSION

RESULT

`count('a,b,c', ',')`

`2`

two delimiters

`dcount('a,b,c', ',')`

`3`

three fields

`dcount('a,,c', ',')`

`3`

the empty middle field still counts

`dcount('a', ',')`

`1`

no delimiter, one field

`dcount('', ',')`

`0`

not 1

`count('aaaa', 'aa')`

`2`

matches do not overlap

`dcount()` **of an empty string is zero, not one.** Everywhere else an empty string behaves like a single empty field, so `for i = 1 to dcount(x, @fm)` is the correct idiom precisely **because** it does nothing when `x` is empty. Code that assumes at least one field and indexes `field(x, ',', 1)` without checking will read an empty value rather than fail.

`count()` **does not count overlapping matches.** `count('aaaa', 'aa')` is `2`, not `3`: once a match is taken, scanning resumes after it. This matters when the substring can overlap itself.

Delimiters may be more than one character — `dcount('a::b', '::')` is `2`.

Finding a position

`index(string, substring, occurrence)`

Returns the character position at which the *occurrence*-th appearance of *substring* starts, or `0` if there is no such occurrence. **The occurrence argument is not optional.**

EXPRESSION	RESULT
<code>index('banana', 'an', 1)</code>	2
<code>index('banana', 'an', 2)</code>	4
<code>index('banana', 'an', 3)</code>	0 — there is no third
<code>index('banana', 'z', 1)</code>	0
<code>index('banana', 'an', 0)</code>	0

Because a miss and a zeroth occurrence both return 0, and position 0 is not a valid position, a simple `if index(...)` then is a safe test for "was it found".

`col1()` AND `col2()`

```
col1()
col2()
```

After a `field()` call, these give the positions **either side** of the field that was extracted: `col1()` is the position of the delimiter before it and `col2()` the position of the delimiter after it.

For `field('alpha,beta,gamma', ',', 2)`, which returns `beta`:

<code>col1()</code>	6 — the comma before <code>beta</code>
<code>col2()</code>	11 — the comma after it

For the first field `col1()` is 0, since nothing precedes it. **When the field is not found, both are 0** — which is also what they read before any `field()` has been called, so they cannot distinguish "not found" from "not asked". Test the result of `field()` itself.

`col1()` and `col2()` belong to the running program, and every called program starts with both at zero. A subroutine that calls `field()` changes its own pair, not the caller's.

Extracting part of a string

BY POSITION

The substring operator takes a start and a length:

```
string[start, length]
```

EXPRESSION ON 'ABCDEFGH'	RESULT	
s[2, 3]	bcd	
s[2, 99]	bcdefgh	a length past the end is not an error
s[9, 2]	empty	a start past the end gives nothing
s[0, 3]	abc	a start below 1 is treated as 1
s[-3, 2]	ab	so is a negative start

The one-argument form is the rightmost characters, not the character at that position.

```
s[3]      ;* fgh - the last three characters of 'abcdefgh'
```

This reads like an array subscript and is not one. It is the commonest misreading of SD BASIC by someone arriving from another language.

The substring form can also be assigned to, replacing that stretch in place:

```
z = 'a,b,c'
z[2,1] = 'Z'      ;* z is now 'aZb,c'
```

BY DELIMITER

```
field(string, delimiter, occurrence {, count})
fieldstore(string, delimiter, start, count, replacement)
```

field()	returns the <i>occurrence</i> -th delimited field. With <i>count</i> , returns that many fields including their delimiters
fieldstore()	replaces <i>count</i> fields starting at <i>start</i> . A count of zero inserts instead of replacing

EXPRESSION	RESULT
field('a,b,c', ',', 2)	b

EXPRESSION	RESULT
<code>field('a,b,c', ',', 2, 2)</code>	<code>b,c</code>
<code>field('a,b,c', ',', 9)</code>	<i>empty</i>
<code>fieldstore('a,b,c', ',', 2, 1, 'X')</code>	<code>a,X,c</code> — replaced
<code>fieldstore('a,b,c', ',', 2, 0, 'X')</code>	<code>a,X,b,c</code> — inserted

An *occurrence* below 1 is treated as 1.

`substrings(array, start, length)` applies the substring operator to **every element** of a dynamic array:
`substrings('abcd' : @vm : 'efgh', 2, 3)` gives `bcd` and `fgh`.

Trimming

```
trim(string)
trimb(string)
trimf(string)
trim(string, character {, mode})
```

The one-argument forms work on spaces:

EXPRESSION ON	RESULT
<code>' A B '</code>	
<code>trim()</code>	<code>a b</code> — leading, trailing and repeated inner spaces
<code>trimb()</code>	<code>a b</code> — trailing only
<code>trimf()</code>	<code>a b</code> — leading only

The three-argument form works on any character and takes a mode letter. **All nine, on** `'xxaxxbxx'` **trimming** `x`:

MODE	MEANING	RESULT
a	remove every occurrence	<code>ab</code>
b	leading and trailing only	<code>axxb</code>
c	compress repeats, leave the ends	<code>xaxbx</code>
d	as <code>trim()</code> — spaces, both ends and compressed inside	—

MODE	MEANING	RESULT
e	as <code>trimb()</code> — trailing spaces	—
f	as <code>trimf()</code> — leading spaces	—
l	leading only	<code>axxbxx</code>
t	trailing only	<code>xxaxxb</code>
r	both ends and compress repeats inside	<code>axb</code>

Omitting the mode gives `r`, not `b`: `trim(t, 'x')` is `axb`. If you only want the ends stripped, you must say `'b'` — this is the difference between `a b` and `a b` on real data and it is easy to miss.

Modes `d`, `e` and `f` ignore the character argument entirely and act on spaces, so the character you pass with them is never read.

For every other mode the character must not be empty. A null second argument stops the program with "Null string as second argument to TRIM()" — so a `trim(x, sep)` where *sep* came from data needs checking before the call, not after.

`CROP()`

```
crop(string)
```

Removes **trailing empty fields, values and subvalues** — the debris left by deleting the last element of a dynamic array. `crop()` of a value list `a`, `b`, empty, empty gives just `a`, `b`. It does not touch spaces.

Building strings

```
space(count)
str(string, count)
```

`space()` *count* spaces. `space(0)` is empty

`str()` *string* **repeated** *count* times

`str()` **repeats, it does not pad.** `str('ab', 3)` is `ababab`, six characters, not `ab` padded to three. To pad, concatenate `space()` and take a substring, or use `fmt()` — see SD Basic - Data Conversion. `str('ab', 0)` is empty.

Strings are joined with the `:` operator, and `:=` appends in place:

```
line = 'a' : ',' : 'b'
line := ',c'
```

`cats(array1, array2)` joins two dynamic arrays **element by element** rather than end to end, and `splice(array1, string, array2)` does the same with a separator between each pair:

EXPRESSION ON <code>A, B</code> AND <code>1, 2</code>	RESULT
<code>cats(p, q)</code>	<code>a1</code> and <code>b2</code>
<code>splice(p, '-', q)</code>	<code>a-1</code> and <code>b-2</code>

Changing case

```
upcase(string)
downcase(string)
swapcase(string)
```

EXPRESSION ON <code>'ABC1'</code>	RESULT
<code>upcase()</code>	<code>ABC1</code>
<code>downcase()</code>	<code>abc1</code>
<code>swapcase()</code>	<code>AbC1</code>

Digits, punctuation and mark characters are left alone.

`fold()` **is not a case function.** Despite the name it wraps text into lines — see *Wrapping text into lines* below. The case-insensitive comparison people reach for `fold` expecting is `compare()` or, for whole files, the no-case file option described under SD Basic - File Handling.

Continued in

SD Basic - String Matching and Replacing — replacing, wrapping, comparing, pattern matching, quoting and phonetic matching.